LLM-Driven Alpha Factor Mining

Yujun Yao
New York University CDS
yy4107@nyu.edu

Haotong Wu
New York University CDS
hw2933@nyu.edu

1 Introduction

Identifying alpha factors is a fundamental problem in quantitative investing, as they capture the outperformance of assets relative to a benchmark (e.g., S&P 500). Traditional factor design relies on expert knowledge, limiting scalability and systematic exploration. While methods such as Genetic Programming expand the search space, they often suffer from low interpretability and unstable performance. Recent advances in large language models (LLMs) enable automated generation of factor expressions through natural language instructions and iterative feedback. Prior work shows that LLM-based systems can effectively assist in discovering and refining financial strategies [3, 2].

However, the main bottleneck lies in the computational pipeline after generation, including both factor computation and factor evaluation. Each candidate factor must be computed on historical data and assessed using financial metrics, and large-scale backtesting quickly becomes computationally expensive as the number of factors grows. Therefore, this project focuses on improving the computational efficiency of the LLM-driven alpha mining pipeline, especially the factor computation and evaluation stages.

2 Methodology

2.1 System Pipeline and Data

The overall workflow of our project consists of five stages: data loading, LLM-based factor generation, factor computation and backtesting-based evaluation. First, we construct a stock-time panel using market data to provide input features for factor calculation. Next, we generate candidate factor expressions using an LLM, which are then analyzed into executable formulas. The system then calculates factor values across the entire panel and evaluates them by matching each factor with future returns. This design separates factor generation from factor evaluation, making it easier to reuse the same set of candidate factors across different optimization experiments.

Our experiments use historical market data for S&P 500 constituents. The processed dataset spans 752 trading days and 483 stocks, and includes the standard market fields used by our factor language. The data are stored in a local cache file and reused throughout the project, ensuring that runtime comparisons are conducted under the same workload.

2.2 Factor Expression Language and LLM Generation

We define alpha factors as symbolic expressions over a fixed operator library and a set of raw market data fields. Each factor maps a panel of stock features to a cross-sectional signal, producing one value per stock per day. The available data fields are: `open`, `high`, `low`, `close`, `volume`, `vwap`, `returns`, and `adv20` (20-day average daily volume). Operators are divided into four categories, as summarized in Table 1.

Table 1: Operator Library

Category	Operators
Time-series	`ts_mean`, `ts_std`, `ts_delta`, `ts_max`, `ts_min`, `ts_corr`
Cross-sectional	`rank`, `zscore`
Mathematical	`sign`, `log`, `abs`, `power`
Arithmetic	$+$, $-$, $\times$, $\div$

Time-series operators take a data field and a lookback window size d as arguments and compute rolling statistics along the time axis for each stock. Cross-sectional operators standardize values across stocks on each day. Factors are constructed by composing these operators into nested expressions, such as `rank(ts_corr(close, volume, 10))`, which computes the cross-sectional rank of the 10-day rolling price-volume correlation.

For the generation part, we use the Claude API (Sonnet 4) to generate candidate alpha factors. Each call uses a structured prompt that specifies the permitted operators, data fields, output format, and few-shot examples with economic rationales. To improve diversity, generation is organized around 12 thematic signal categories, such as momentum, mean reversion, intraday structure, volatility regimes, and cross-signal interactions.

All generated expressions are parsed into an Abstract Syntax Tree (AST) using Python's `ast` module. A whitelist checker rejects expressions with disallowed operators, unrecognized fields, or attribute access. Valid expressions are then passed to the redundancy and complexity filters described below.

2.3 Computational Optimization

The computational pipeline consists of two main stages: factor computation and factor evaluation.

In the factor computation stage, symbolic factor expressions are transformed into numerical matrices over the stock-time panel. In the evaluation stage, the predictive power of each factor is assessed by computing metrics such as Information Coefficient (IC), Sharpe ratio, and maximum drawdown through backtesting. In the baseline implementation, factor computation daccounts for most of the total running time (taking up 95%), so we prioritize optimizing the computation stage first. After the optimization of the first stage, the bottleneck is transferred to the evaluation stage, guiding our second stage optimization.

2.3.1 Stage 1: Factor Computation

In the baseline implementation, all time-series and cross-sectional operators are implemented as explicit row-by-row Python loops over the 752-day panel, resulting in $O(T \cdot N)$ iterations per operator call.

Opt	Technique	Key Idea
Baseline	Python loops	Row-by-row iteration over the stock-time panel for all time-series and cross-sectional operators, resulting in $O(T \cdot N)$ Python-level iterations per operator call.
Opt1	Python-level optimizations	Reduce Python overhead by simplifying repeated operations while preserving the baseline structure.
Opt2	Pandas vectorization	Replace Python loops with `DataFrame.rolling()` and `rank()`, reducing execution to optimized library operations.
Opt3	NumPy + Bottleneck	Convert panel data to contiguous arrays and use `bottleneck` moving-window functions to reduce DataFrame indexing overhead.
Opt4	Numba JIT kernels	Compile time-series operators with fused single-pass sliding-window kernels, such as `ts_corr`, to reduce intermediate arrays.
Opt5	Subexpression caching	Parse factor expressions into ASTs and reuse shared subexpressions, reducing redundant computations.
Opt6	Parallel execution	Use `ThreadPoolExecutor` for factor-level parallelism while disabling Numba internal threading to avoid contention.

Table 2: Progressive optimization pipeline for factor computation.

2.3.2 Stage 2: Factor Evaluation

The baseline evaluation implementation iterates over 752 trading days in Python, using `scipy.stats.spearmanr` per day for IC computation, and performing per-day rank-and-select loops for portfolio construction and turnover estimation.

We replace all three components with fully vectorized implementations. Spearman IC is computed as the Pearson correlation of cross-sectional ranks, using `DataFrame.rank(axis=1)` on the full panel. Long-short returns are built from boolean quantile masks applied to all days at once. Turnover is computed from element-wise differences between consecutive holdings matrices.

To further reduce redundancy, the three components are fused to share a single factor ranking computation. In addition, factor-level evaluation is parallelized using `ThreadPoolExecutor`.

Overall, these optimizations reduce total evaluation runtime from 191.40s in the baseline to 35.82s in the full optimized pipeline.

2.4 Factor Generation Strategy

Beyond computational efficiency, we aim to generate meaningful alpha factors guided by evaluation metrics. Inspired by previous research, we implement three techniques to guide the LLM toward more effective alphas: complexity constraints, AST redundancy filtering, and a mutation feedback loop.

Complexity constraints. Each factor is evaluated along three dimensions: expression length, parameter count (i.e., number of window sizes or thresholds), and the number of distinct raw features. Factors exceeding predefined thresholds are rejected: (1) sym_len > 200 characters, (2) depth > 5 nesting levels, (3) $n_{\text{fields}} > 6$ distinct raw features, and (4) complexity_score > 80. These thresholds are motivated by findings in [1], which show that removing complexity

constraints causes the most severe degradation among all ablation settings, with an 8.44% decrease in annualized return. This suggests that overly complex expressions tend to overfit historical noise.

AST redundancy filter. We normalize each factor's Abstract Syntax Tree (AST) by replacing all numeric constants with a placeholder value, producing a structural signature that abstracts away parameter choices. Two factors sharing the same signature differ only in numeric arguments—for example, `rank(ts_delta(close, 5))` and `rank(ts_delta(close, 3))`—and are treated as structural duplicates. From each duplicate group, only one representative is retained.

Mutation feedback loop. To improve factor quality beyond one-shot generation, we adopt an iterative evolutionary framework applied to the factor pool over multiple rounds. Each round combines two complementary operators: **crossover**, which targets top-performing factors and prompts the LLM to generate structurally distinct variants or hybrid factors; and **mutation**, which targets bottom-performing factors by submitting each expression alongside its evaluation metrics and instructing the LLM to diagnose failure modes and generate improved variants. Newly generated candidates must pass syntactic validation, complexity constraints, and AST redundancy filtering before evaluation. Candidates scoring above the current pool median are merged back into the factor pool, progressively improving mean factor quality while maintaining structural diversity.

2.5 Factor Evaluation and Scoring

We evaluate each alpha factor by aligning its cross-sectional signal with next-day returns. Let f_t denote the factor values on day t and r_{t+1} denote the corresponding next-day returns, both as vectors of length $N = 483$. The evaluation metrics are summarized in Table 3.

Table 3: Factor Evaluation Metrics

Metric	Description
IC	Average daily Spearman rank correlation between f_t and r_{t+1}, measuring predictive power.
ICIR	Mean IC divided by the standard deviation of the IC time series, measuring signal stability.
Sharpe	Annualized risk-adjusted return of a long-short portfolio that buys the top decile and shorts the bottom decile by factor value.
MDD	Maximum drawdown of the long-short portfolio, measuring the worst peak-to-trough decline. Lower drawdown is preferred.
Turnover	Average daily change in long-short holdings, used as a proxy for trading cost. Lower turnover is preferred.

To obtain a single ranking criterion, each metric is first normalized using min-max scaling. Since lower drawdown and turnover are preferred, we use $-$MDD and $-$Turnover in the composite score:

$$\text{Score} = 0.15 \cdot \text{norm}(\text{IC}) + 0.35 \cdot \text{norm}(\text{ICIR}) + 0.30 \cdot \text{norm}(\text{Sharpe}) + 0.15 \cdot \text{norm}(-\text{MDD}) + 0.05 \cdot \text{norm}(-\text{Turnover}).$$

3 Experiments

All experiments are conducted on the same cached S&P 500 stock-time panel described in Section 2.1. To ensure fairness in comparisons and to save time in generating factors using the LLM, we used the same set of generated factor pools across all experiments. The experiments evaluate two aspects of the system: computational efficiency and factor quality. For efficiency, we measure the runtime of both factor computation and factor evaluation. For factor quality, we rank generated factors using the composite score introduced in Section 2.5.

3.1 Runtime Performance

Table 4 reports the runtime of each optimization stage for factor computation. The baseline is very slow because time-series and cross-sectional operators are executed through Python-level loops over dates and stocks, creating large interpreter overhead.

Opt1 only provides a slight improvement because it reduces Python overhead without changing the overall computational structure. The main speedup starts from Opt2, where Pandas vectorization replaces explicit Python loops with optimized routines such as `DataFrame.rolling()` and `rank()`. Opt3 uses contiguous NumPy arrays and Bottleneck moving-window functions to reduce DataFrame indexing overhead. Opt4 applies Numba JIT compilation to time-series kernels, Opt5 caches shared subexpressions across factors, and Opt6 uses `ThreadPoolExecutor` for factor-level parallelism while avoiding contention with Numba's internal threads.

The results show that the largest improvement comes from removing Python-level loops from the factor computation stage. Opt2 reduces runtime from 1819.37s to 10.37s, indicating that vectorization is the first major turning point. Later optimizations continue to improve performance, with Opt6 Full reaching 0.68s, corresponding to a 2680.0$\times$ speedup over

Method	Comp. (s)	Speedup
Baseline	1819.37	1.00×
Opt1	1814.62	1.00×
Opt2	10.37	175.4×
Opt3	3.66	497.5×
Opt4	3.37	539.4×
Opt5	3.21	567.0×
Opt6	0.87	2099.2×
Opt6 Full	0.68	2680.0×

Table 4: Factor computation runtime.

Method	Eval. (s)	Speedup
Baseline	191.40	1.00×
Opt1	158.36	1.21×
Opt2	197.35	0.97×
Opt3	190.80	1.00×
Opt4	195.51	0.98×
Opt5	188.24	1.02×
Opt6	162.34	1.18×
Opt6 Full	35.82	5.34×

Table 5: Factor evaluation runtime.

the baseline. This confirms that factor computation is the main bottleneck in the original implementation and benefits strongly from vectorization, compiled kernels, caching, and parallelism.

For factor evaluation, most intermediate optimization stages do not substantially reduce runtime because they mainly target factor computation. The main evaluation-side improvement appears in Opt6 Full, where Spearman IC, long-short portfolio returns, and turnover are vectorized and fused to share intermediate ranking results. As a result, evaluation runtime decreases from 191.40s in the baseline to 35.82s in Opt6 Full, a 5.34× speedup.

3.2 Factor Quality Analysis

Beyond runtime, we evaluate whether the generated factors produce meaningful trading signals. After syntax validation, complexity filtering, AST redundancy filtering, and mutation feedback, valid factors are ranked using the composite score. After three rounds of enhance and mutation, we increase the factor size by 30.5%, enhancing the median score by 5.8% and top score by 8.1%. The top-ranked factor is

$$\begin{aligned}\text{factor} = {} & \texttt{rank(vwap / close - 1)} \\ & \times \texttt{(rank(volume / adv20) * rank(ts\_std(returns, 5))} \\ & \quad \texttt{+ ts\_delta(rank(volume / adv20), 3))} \\ & \times \texttt{rank(ts\_corr(volume, abs(returns), 10))}\end{aligned}$$

with a score of 0.9419, IC mean of 0.0167, ICIR of 0.2035, and Sharpe ratio of 2.658. This factor combines intraday price-location information, abnormal trading volume, short-term return volatility, and volume-return interaction, suggesting that the generated expressions can capture robust price-volume dynamics than simpler one-term factors while remaining interpretable.

4 Conclusion

In this project, we built an LLM-driven alpha factor mining pipeline covering factor generation, validation, filtering, computation and backtesting-based evaluation. The main goal is to improve the efficiency of the computation and evaluation stages after factor generation.

Experiments show that the full optimized pipeline reduces factor computation runtime from 1819.37s to 0.68s, and reduces factor evaluation runtime from 191.40s to 35.82s. These improvements mainly come from vectorization, array-based operations, Numba JIT kernels, subexpression caching, fused evaluation, and parallel execution. We also proved that strategies including complexity constraints, redundancy checks, and Feedback loop can help generate alphas with better performance. Future work could extend the system to larger datasets and a more automated LLM feedback loop.

References

[1] Jun Han, Shuo Zhang, Wei Li, Zhi Yang, Yifan Dong, Tu Hu, Jialuo Yuan, Xiaomin Yu, Yumo Zhu, Fangqi Lou, Xin Guo, Zhaowei Liu, Tianyi Jiang, Ruichuan An, Jingping Liu, Biao Wu, Rongze Chen, Kunyi Wang, Yifan Wang, Sen Hu, Xinbing Kong, Liwen Zhang, Ronghao Chen, and Huacan Wang. Quantaalpha: An evolutionary framework for llm-driven alpha mining. *arXiv preprint arXiv:2602.07085*, 2026.

[2] Zhizhuo Kou, Holam Yu, Jingshu Peng, and Lei Chen. Automate strategy finding with llm in quant investment. *arXiv preprint arXiv:2409.06289*, 2024.

[3] Yanlong Wang, Jian Xu, Hongkang Zhang, Shao-Lun Huang, Danny Dongning Sun, and Xiao-Ping Zhang. Factorminer: A self-evolving agent with skills and experience memory for financial alpha discovery. *arXiv preprint arXiv:2602.14670*, 2026.